

Clingonia

Introducing an Enhanced Version:
Expanded Features, Improved Usability,
Cross-Platform Robustness

Adrian Henze and Max Vogel

Final Report
by MadMotion

Railway Scheduling in Flatland
University of Potsdam
Germany
March 2026

Table of Contents

1 Introduction 4
2 New Features 6
2.1 Global Time Limit 6
2.2 Speed 6
2.3 Malfunction 7
3 Quality-of-Life Improvements 8
3.1 Clingo Options 8
3.2 Clingo Python API 8
3.3 Printing Clingo's Last Answer 8
3.4 True/False Switches 9
3.5 macOS Support 9
3.6 Different Starting View for Modify Mode 9
3.7 Zoom Levels 9
4 Optimization 10
4.1 Build Canvas 10
4.2 Environment Canvas 10
4.3 GIF Canvas 11
4.4 Dynamic Path Labels 11
5 Error Handling 12
5.1 Build Dimensions 12
5.2 Empty Inputs 12
5.3 Answer Number 12
6 Bug Fixes 12
6.1 Trains on Tracks 12
6.2 Environment Preservation 12
6.3 Discard Changes Window 13
6.4 Frame-to-Frame Counter 13
6.5 Linux 13
7 Updates 14
7.1 Flatland Update 14
7.2 Overviews and Documentation 14
8 Future Research 15

Abstract. Clingonia is a comprehensive GUI software to generate, build, modify, save, and solve environments using Clingo - powered by the Answer Set Programming paradigm. This report documents Clingonia's evolution during the winter semester of 2025/2026, detailing its architectural refinements and functional enhancements. These encompass new features that were added to Clingonia, various quality-of-life improvements, crucial canvas optimizations, necessary error-handling strategies as well as bug fixes, and vital updates that keep Clingonia operational. For prospective course participants, an outlook is provided with potential future research opportunities.

1 Introduction

Last year, during the Railway Scheduling Research Internship program, we created our software, Clingonia, introducing it as a graphical user interface (GUI) for managing and visualizing Flatland environments. Back then, it was already in great demand, because not only we, but also other student researchers, were looking for a better way to create, manage, and solve Flatland environments. So, we had to ask ourselves how we could transform the existing research workflow into something better. We thought it through and came up with clear objectives, creating a vision. And that vision is embodied by Clingonia.

Prior to Clingonia, there was no software that provided a proper GUI for conducting research in this course. Users had to work closely with the terminal, executing commands to run experiments. They needed to look through a lot of files and folders to script and modify their parameters, so that the scenario was shaped the way they wanted it. This meant that, instead of managing only the environments, users also had to manage the surrounding filesystem. Moreover, they had to hope that the setup with their custom parameters would work, because there was no feedback on the input – a lack of transparency and robustness. Building a custom environment suffered under the same conditions, because users also had to hardcode the entire environment in a way that is unintuitive and thus prone to errors. This was a tedious task, and there was no way around it. And lastly, to visualize custom environments, there was nothing but separate, inefficient tools, like pen and paper. The change was undoubtedly necessary.

Today, Clingonia is a comprehensive GUI software with all the basic functionality users need for their research and now even includes advanced features such as speed and malfunctions. The terminal, instead of being the main workspace, now only serves to launch Clingonia and to get more information about what is happening in the background, allowing full transparency. Users do not need to look through the filesystem anymore, because everything is now laid out in a structured and clear manner right in the software, creating a workspace that contains all the important features without the necessity of a separate tool. This robust workspace can provide instructive error messages that tell our users exactly what input is expected, which allows them to make adequate adjustments. Clingonia provides an easy-to-use build menu and for that reason, building and modifying environments was never easier in this course: users can simply select and place tracks with the mouse without worrying about the details. There is no longer any need for tedious scripting, and users do not even need to understand the Answer Set Programming (ASP) Language to build environments. Finally, with Clingonia, users can visualize both their environments and their solutions in a qualitative and Flatland-like manner. Solutions can be represented in several ways: as a timetable and map with numbered paths, as an animation or frame-by-frame presentation, and also in the traditional way in the terminal.

User feedback confirmed the success of our mission. Clingonia was created for all experience levels in this research field: no matter whether one is a beginner who is not even familiar with ASP, or an advanced user working on a

Master's thesis. Since our project was in great demand, users requested access to our software as soon as possible. For this reason, we welcomed our first users as soon as we published the early-access version. As a result, other groups were already able to present their research using visualizations created with our software. One year later, student researchers ran Clingonia for the entire semester, supporting and enhancing their research process significantly and hence, their project results.

Now, we have chosen to come back because, on the one hand, Clingonia still lacked certain features that we could not implement last time. On the other hand, Clingonia had to go through a crucial Flatland update and besides some issues that needed to be fixed, there was still a lot of room for optimization. Thus, this semester, we made Clingonia easier to use as well as more enjoyable, accessible, and robust.

2 New Features

Our new features expand Clingonia's research capabilities. First, the course introduced a new common fact that we needed to implement. In addition, we knew that there were two advanced features still missing, namely speed and malfunctions.

2.1 Global Time Limit

Since the previous release, the course introduced a new common fact, namely the global time limit predicate. It has the form *global(MaxTime)*. Because it defines the environment's time horizon and is therefore a fundamental predicate rather than an advanced feature, it was one of the first things we implemented.

The parser now looks for this predicate when loading a file. When saving an environment, Clingonia writes this predicate to the file. We chose a default value of 100 for the global time limit.

In the user interface (UI), an input field for the global time limit was introduced to the generation and build parameter menus, as shown in Figure 1. The user input is displayed in the parameter overview at the end of the generation, building, and loading procedures.

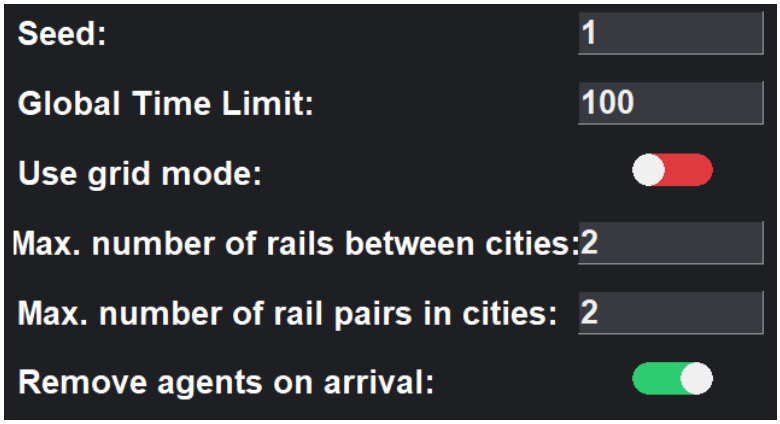


Fig. 1: Global Time Limit and Switches

2.2 Speed

The first missing feature we implemented was train speed. Now, users have the option to assign individual speeds to their trains. The default speed is 1.

To randomize the assigned speeds, users can use the speed map. Last year, we already implemented the corresponding UI component, but it had no functionality. Previously, all trains had a fixed speed of 1. It is now fully operational

and can be used to randomly assign custom speeds with user-defined probabilities. The speed generation is driven by the user-defined environment seed. If the probabilities do not add up to 1, Clingonia automatically normalizes them. Because the speed map requires the actual speed rather than its inverse, our software converts the input accordingly, as the course specification operates on inverse speed values.

When loading environment scripts, the speed predicates in the file that conform to the common fact format of our course are parsed and loaded into Clingonia.

We removed the speed map input from the build and modify parameter pages. The manually placed trains now have their inverse speeds assigned from a dedicated input field when configuring the specific train, just as we already handle the earliest departure and latest arrival settings. Also, similar to the other two parameters of each train, we added a setting to the *Change All* menu that allows the user to assign the same inverse speed value to all trains at once. The speed map input in the generation menu remains unchanged.

In all environment overviews displayed at the end of the build, modify, generation and loading procedure, Clingonia now shows the assigned inverse speed parameters as an additional column in the train table. The generation menu still contains the speed map in its parameter section of the overview screen.

We modified the result page to show the inverse speed of each train along with its ID, position, and direction in the train list.

2.3 Malfunction

The second previously missing feature was the malfunction generation. Users can now randomly generate malfunctions for their environments.

While the speed generation is driven by the environment seed, the malfunction generation is intentionally independent of that seed. To ensure reproducibility, we provide a dedicated switch utilizing the user-defined seed.

Regarding the implementation, Clingonia first checks whether the user entered a non-zero malfunction rate to determine whether malfunctions should be generated. It then verifies whether reproducibility is enabled. If malfunction generation is enabled, the program randomly generates them for all trains up to the user-defined global time limit. At each timestep, a malfunction is triggered with probability equal to the specified malfunction rate. It then randomly chooses a train and a duration between the specified minimum and maximum values. Finally, Clingonia writes the generated malfunction to a dedicated file, *malfunction_tmp.lp*, which contains all malfunction predicates.

If users are not satisfied with the generated malfunctions, they can directly customize the corresponding file. This file is automatically added to the Clingo command when running the simulation. It exists only while Clingonia is running and only if the malfunction rate is non-zero.

The malfunction parameter settings for rate, minimum duration, and maximum duration remain unchanged in the UI.

3 Quality-of-Life Improvements

At the beginning of this semester, Clingonia offered considerable room for improvement. The following changes were implemented to make Clingonia more enjoyable, accessible, and robust.

3.1 Clingo Options

The final feature added right after the previous winter semester was a rudimentary mechanism for adding options to the Clingo command. Now, we have fully integrated a dedicated entry field directly into the user interface, as illustrated in Figure 2. This allows the users to add their custom options without manually modifying files anymore.

To make this feature more robust, we extracted all the current options from Clingo's help pages and tested each and every one of them with Clingonia. Only a few were not compatible with our parser. We created a blacklist for those options, which is why Clingonia does not add them to the Clingo command.

3.2 Clingo Python API

Clingonia was previously tied to a local Clingo installation that invokes Clingo as a separate process. We realized that environments could be solved directly using Clingo's Python application programming interface (API). Now, this local installation is no longer required.

To implement this, we split the *run_clingo* function into two components. The original function was retained as *run_clingo* for the Clingo command line interface (CLI), while a second function, *run_clingo_api*, was introduced for the Python API. The timer thread logic was extracted into a separate function, as it is shared by both implementations. The API-based solver can be invoked by entering 'API' in the Clingo path field (see Figure 2).

Moreover, our software uses the Python API as a fallback when the user requests the CLI but the separate installation cannot be found. Clingonia always notifies the user in the terminal when the API is used. Since the Clingo Python API has a different output format than the Clingo CLI, the API output is converted to closely match the CLI output format.

3.3 Printing Clingo's Last Answer

A student from another group reported that Clingonia did not show Clingo's last satisfiable answer when the requested answer number exceeded the available answers. For users, this meant manually adjusting the requested answer number and waiting for Clingo to compute each time. Now, Clingonia prints this last answer (see Figure 2), eliminating the need for manual trial and error.

To address this issue, we implemented a new function called *print_last_clingo_answer*, which identifies the last computed answer and prints it to the terminal if that answer exists; otherwise, it notifies the user that Clingo returned UNSATISFIABLE.

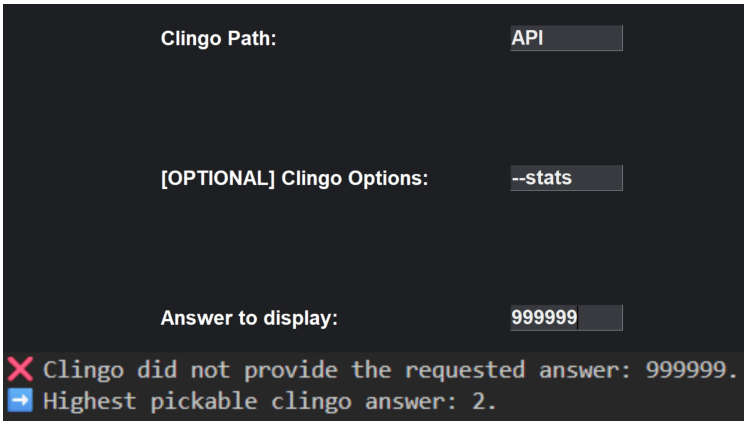


Fig. 2: Clingo Improvements: API, Options, Answer.

3.4 True/False Switches

We replaced entry fields for binary parameters, such as *Use grid mode* or *Remove agents on arrival*, with simple toggle switches, also shown in Figure 1. That simplifies and speeds up the initial setup of the environment, as users no longer need to type *True* or *False*. This also reduces the possibility of errors occurring from the user's input.

3.5 macOS Support

On macOS systems, the panning of the environments was previously not working. Users had to zoom in and out to navigate the environment. Now, we adjusted the macOS input bindings, providing a different button assignment for these systems to circumvent this problem, ensuring correct panning functionality.

3.6 Different Starting View for Modify Mode

We noticed that it is not always useful to start zoomed into the top left corner of the environment when using the modify mode. For example, when modifying a randomly generated environment, the corners are usually empty. This empty space might suggest it did not load the environment properly, or it might just be inconvenient to have to move to the spots where the user wants to make changes. This is why Clingonia now starts zoomed out and centered on the environment, so the user can see the entire layout of their project and simply zoom into the desired sections. There is no need to search for their environment anymore.

3.7 Zoom Levels

To accommodate the different starting positions in the build and modify mode, as well as an initial attempt at optimizing the zoom feature, we adjusted the

minimum and maximum zoom levels to scale with the magnitude of the larger dimension.

4 Optimization

While working on other improvements, we noticed that the performance of the build canvas and zoom function on all canvases was substandard in larger environments, so we decided to fix that. In the following section, we will address the changes made to each canvas type, starting with the canvas in the build menu. This is followed by the environment canvas, used in the overview screens at the end of the generation and build menu, as well as the main and result menu, and lastly the GIF canvas, which displays the animation of the solution.

4.1 Build Canvas

For the build canvas, the main change was to reduce the number of actions the program had to perform for each placed or removed object on the canvas.

In its previous version, the canvas removed all trains and stations before redrawing all of them at once when a new one was placed. This was changed to only affect the cell in which the new object was placed.

In connection with that, the train list, displayed next to the canvas, now only appends the new train at the end of the list and does not, like in earlier versions, redraw the entire list with all its elements, such as buttons and labels, after the new train has been added to the underlying data structure.

We also introduced caching of images to improve the zoom performance, as this reduces the number of times the images have to be rendered when the same zoom levels are visualized repeatedly.

4.2 Environment Canvas

Our approach to improving this canvas's zoom performance is based on two main strategies: First of all, our program calculates which portion of the image is visible at the current zoom level. Then it creates a copy of the original image and cuts out that portion, which is subsequently rendered as its zoomed-in version.

The second part of our strategy is to render the image as a low-quality version with a faster algorithm, and display this image first, so the user can pan or zoom in further if they want to see more details. If no further zoom actions are called, Clingonia opens a background thread and renders a high-quality version of the currently displayed image. This allows the user to still move the image and does not force them to wait for the program to finish. Once the high-quality image is ready, it replaces the low-quality image.

This approach required us to also rework the panning function. Previously, we simply moved the image object on the canvas; this is not possible anymore, as we only have the cut-out section of the image available. The change we implemented is to simply render the new slice of the image and append it to the already

existing render while caching the now off-screen portion. As the section that has to be rendered when moving the image is comparably small, the performance of this method is nearly indistinguishable from the former pan function.

4.3 GIF Canvas

We tried to implement the same new zoom feature from the environment canvas for the GIF canvas, but this did not work, as it is a sequence of images. An alternative approach was necessary to improve the performance of the zoom function for this canvas.

We ended up increasing the step size when zooming and only having a few discrete zoom levels. This allows the user to get a more detailed view of their environment faster, even with the previous, slower zoom function.

While the user is zooming, we use a faster resampling algorithm, and when no zoom action is happening, the program uses a high-quality algorithm when replacing the individual frames during animation.

To accommodate different environment sizes, we dynamically add zoom levels with larger magnification and simultaneously remove small magnifications when the size of the environment is above certain thresholds. This skips smaller steps that do not increase the level of detail significantly in large environments and allows for a consistent zoom over all environment sizes without having more render actions.

4.4 Dynamic Path Labels

We decided to make the font size of the paths displayed on the result canvas dynamic to balance readability and overlap prevention. The font size is now double the previous font size when only one path is displayed, and is reduced again by half if multiple paths are displayed at once. This is illustrated in Figure 3.

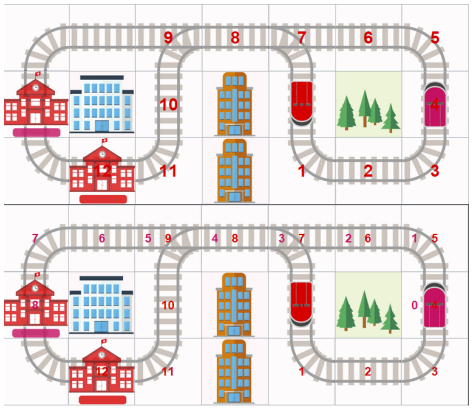


Fig. 3: Dynamic Paths: Font size based on number of displayed paths.

5 Error Handling

5.1 Build Dimensions

During testing, we noticed that Clingonia accepts negative values for the dimensions of the environment when initializing the build and modify modes. As this does not make sense physically or technically, we limited the accepted values to positive integers larger than zero.

5.2 Empty Inputs

To improve the clarity of our program, we decided to change how our software handles empty input fields. Previously, we allowed the users to leave the inputs on the generation and build parameter pages empty, and Clingonia would fill them with default values. To make sure the user always knows why something is happening, we decided it is clearer to require users to fill in empty input fields themselves, even if they use the default values we provide as examples. Thus, Clingonia now displays an error message beside the input field if it is left empty when clicking the button to continue to the next menu.

5.3 Answer Number

We had to introduce an upper limit for the answer number in the Clingo parameter menu, as we found that a number larger than 2^{31} leads to an error from Clingo. Thus, we only allow inputs from 0 to $2^{31} - 1$. Zero is allowed as an option to force a rerun of the solver, even if none of the input parameters have changed, for example, if the user made changes to their ASP encodings.

6 Bug Fixes

6.1 Trains on Tracks

In previous Clingonia versions, we frequently encountered a bug where a track would be displayed on top of the train (see Figure 4) if the cell containing a train was modified. This did not affect the functionality of the program, but as it is visually disruptive for the user experience, we fixed this. We now ensure the image display order is always such that tracks are on the lowest level by re-rendering the cells' images in the desired order when an object is placed.

6.2 Environment Preservation

Sometimes, the current user environment displayed in the main menu was overwritten with an empty environment when the user entered the build menu and, without finishing the building process, returned to the main menu via the red arrow buttons. This was caused by saving and loading the current user environment to and from a backup at the wrong times, which caused the temporary

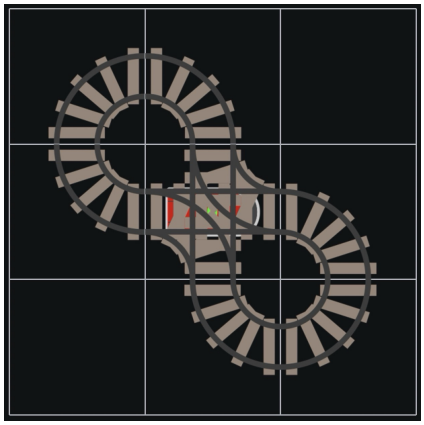


Fig. 4: Train under track.

environment of the build menu to be saved to the backup environment that contained the user's environment. We changed the load and save locations in our workflow to ensure the preservation of the user environment until the build process is properly finished. This should allow the backup to be restored correctly if the build process is aborted.

6.3 Discard Changes Window

The *Discard Changes Window* that requests the user's confirmation if the build process is aborted was not closed properly in previous Clingonia versions. This caused the window to be visible for a split second when changing between different menus. We fixed this by adding a closing command for this window when users select one of the options in the window.

6.4 Frame-to-Frame Counter

When a user viewed a GIF in the Frame-to-Frame view on the results page, the counter tracking the currently displayed time step did not reset properly when the view was closed and reopened. This caused a bug where the GIF started with the first time step in the new sequence, but then jumped to the position of the *old counter + 1*. We fixed this bug by making sure the counter is properly reset to the first time step present in the new image sequence.

6.5 Linux

The feedback from our users provided insights into the performance of our software on Linux systems. Multiple issues occurred on Linux systems that we did not know about until they were reported, as neither of us uses Linux.

The first reported issue referred to Clingo's output format which differs from the format on other platforms. On Linux, the runtime was printed on the same line as the answer number. As a result, the parser failed to recognize the requested answer. Now, we have adapted Clingonia so that its parser operates correctly. Instead of matching a fixed answer header, the parser now checks whether a line begins with the requested answer. This allows additional runtime information on the same line without affecting parsing. Furthermore, Linux's case-sensitive filesystem caused conflicts with our track image files. The issue was caused by uppercase letters following underscores. To resolve this, the track image files were renamed and their naming scheme was translated from German to English.

Lastly, the notification sound was disabled on Linux. Due to the lack of a Linux testing environment, the issue could not be reliably resolved. Error handling related to sound playback was improved to increase overall robustness. Clingonia now behaves more reliably across platforms.

7 Updates

Clingonia underwent several updates to maintain compatibility and ensure continued operation.

7.1 Flatland Update

The primary objective was ensuring compatibility with the latest Flatland version 4.2.2, because previous Flatland versions relied on packages that have since been deprecated.

The core change that made our software compatible with the latest version was the transition from separate train coordinates and direction values to a waypoint-based representation. Flatland introduced waypoints as a concept that combines both attributes. This was implemented as follows: we assigned two waypoints to every agent. The first waypoint stores its start coordinates and start direction and the other one stores its end coordinates and its end direction. The value *None* is assigned to the end direction, because the course's common facts provide no information about it.

7.2 Overviews and Documentation

Due to the implemented changes, we needed to update our parameter overviews, error messages, and help pages. This includes additional usage guidance, rephrased descriptions, and expanded explanations.

Moreover, we refined our GitHub page by documenting the Flatland version, rephrasing certain sections, noting that the Clingo CLI is now optional and that Clingonia runs on Linux, documenting the global time limit and speed facts. And finally, we updated the pictures and known issues.

8 Future Research

Clingonia now reflects the vision we initially defined. However, there remains room for improvement.

The first thing we would like to mention here is that our ASP problem-solving files are sufficient for most environments, but they are far from flawless. Moreover, our problem-solving code does not consider speed or malfunctions. Future student researchers could take on the task of providing a high-quality ASP problem-solving code for the complete system: a robust general strategy for the trains which also deals with various train speeds and the handling of malfunctions.

Closely related to that, future students could also develop optimal example environments so that users can test their problem-solving strategies under controlled conditions and identify specific weaknesses.

Furthermore, Flatland sometimes generates environments larger than requested. Clingonia trims them to match the request, which may cut off part of the railway system. Fortunately, this can be quickly resolved by the user in the modify mode. This strategy may not represent an optimal solution, so perhaps future student researchers will find a way to resolve this issue without requiring manual intervention.

And finally, we noticed that there is currently no way to cancel certain processes that may take up a lot of time. This includes the environment generation and environment building, the simulation run, and the creation of the animation. Introducing such a feature would further improve usability and robustness.